

Incident Report: Telegram Listener Reconnect Storm

Date: 2026-07-28 **Duration:** ~37.5 minutes (13:45-14:23 UTC) **Severity:** Critical – all users’ Telegram listeners disconnected, most stayed dead **Root cause commit:** af12737d **Fixed by:** Rollback to 01a2d913 + 11 hardening fixes (see section 4)

1. Timeline

Time (UTC)	Event
13:45:29	New deployment starts (commit af12737d on production listener)
13:45:30	Old process killed – all 23+ Telegram sessions disconnect simultaneously
13:45:33	First sessions begin reconnecting
13:45:52	First flood-wait rate-limit from Telegram’s DC
~13:46	First AUTH_KEY_DUPLICATED events (4 sessions)
~13:46	First malformed RPC results (4 sessions)
~13:47-14:20	Continuous reconnect cycle: connect -> flood-wait -> disconnect -> reconnect
~13:47-14:20	15 _updateLoop TIMEOUTs across 15 users
~13:47-14:20	42 ensureSignalRow schema failures (separate issue)
~14:20-14:23	Sessions that exhausted all 10 retries go permanently dead
~14:23	Rollback initiated (deploy 01a2d913)

2. What Happened

2.1 Trigger

A routine deployment restarted the Railway listener worker. All 23+ Telegram MTProto sessions were disconnected at once. This is normal – every deployment causes this.

2.2 The Reconnect Storm

When the sessions tried to reconnect, Telegram’s datacenter saw a flood of connections from the same IP and applied **flood-wait rate-limiting** (Sleeping for Xs on flood wait). This is also normal.

However, the code deployed with af12737d had changed reconnect behaviour in 4 ways that turned a routine restart into a 37-minute outage:

Parameter	Old (working) code 01a2d913	New (broken) code af12737d	Impact
Max reconnect attempts	4	10	Full cycle takes 273s instead of 56s
Delay pattern	Escalating: [cooldown, retry, 15s, 30s]	Flat 30s for every attempt	No fast-path for early recovery
Deferred retry after exhaustion	60s deferred retry re-enters the cycle	None – session dead forever	Sessions hit 10/10 and never recover
Malformed RPC result action	Silently ignored (GramJS internal transient)	Triggers requestReconnect	GramJS noise starts a new 10-attempt cycle

2.3 Flood-Wait Noise

10,036 log lines (83% of total) were GramJS Sleeping for Xs on flood wait messages. These are harmless rate-limit pauses from Telegram, but they drowned out real error signals.

2.4 Sessions Affected

- 23 unique user sessions attempted to connect
- 4 sessions hit AUTH_KEY_DUPLICATED (Telegram rejected the old session still held by the previous process)

- **4 sessions** hit malformed RPC results (body type=undefined – GramJS internal transient)
- **15 sessions** hit `_updateLoop TIMEOUT` – the polling loop timing out because reconnects took too long
- **Multiple sessions** exhausted all 10 attempts -> no deferred retry -> permanently disconnected
- **renewAllLeases skipped 43 times** – flood-wait blocked session lease renewal

2.5 Schema Issue (Unrelated)

42 ensureSignalRow failures for `pipeline_ts` column missing from the `signals` table schema cache. This is a separate schema migration issue (fixed by migration `20260724120000_signals_pipeline_ts.sql`), not caused by the reconnect storm but noisy in the log.

3. Commits Involved

3.1 Breaking Commit

`af12737d` `AUTH_KEY_DUP_RECONNECT_DELAY_MS` is now consistently configurable through, `TELEGRAM_AUTH_DUP_RECONNECT_DELAY_MS`, defaulting to `30_000`

Author: PR #47 (fix/auth-key-duplicated-deploy-recovery) **Merged:** via `eafdfca0` into dev **What it changed:**

1. `authKeyDupMaxRecoveryAttempts()` default: **4 -> 10**
2. `authKeyDupReconnectDelaysMs()`: removed escalating delays, replaced with **flat 30s** for every attempt
3. `authKeyDupDeferredRetryMs()`: **removed entirely** – no function, no deferred retry after exhaustion
4. `noteMalformedRpcResult()`: added `requestReconnect()` call – GramJS transient errors now trigger the full reconnect cycle

3.2 Safe Baseline (Rollback Target)

`01a2d913` PR #43 -- feat/auth-fixes-to-main (Jul 23)

3.3 Related Commits (Pre-existing Fixes Already on Staging)

`b3a8f38a` fix: await `requestReconnect` in `_updateLoop TIMEOUT` handler
`ef01e883` fix: break QR login death spiral on `AUTH_KEY_UNREGISTERED`
`991bf6d2` fix: `CHANNEL_INVALID` auto-disable
`406c3d50` feat: Section 6 scale validation

These were already deployed but did not prevent the storm because none of them addressed the 4 regressions in `af12737d`.

4. Fixes Applied

4.1 Fix 1-2: authKeyDuplicatedRecovery.ts – Restore sane defaults

Plain English: When Telegram says “this session is already connected elsewhere,” the listener needs to wait and retry. The broken code tried 10 times with 30-second pauses (4.5 minutes total) and then gave up forever. Now it tries 4 times with shorter pauses that get longer (first retry fast, then 15s, then 30s). If all 4 fail, it waits 60 seconds and tries again instead of dying forever.

- **maxAttempts:** 10 -> **4** (configurable via `TELEGRAM_AUTH_DUP_MAX_RECOVERY_ATTEMPTS`)
- **Delays:** flat 30s -> **escalating [cooldown, retry, 15s, 30s]** – fast recovery when possible, backoff when needed
- **Deferred retry:** restored – **60s** after exhaustion, re-enters the reconnect cycle (`TELEGRAM_AUTH_DUP_DEFERRED_RETRY_MS`)

4.2 Fix 3: userListener.ts – Malformed RPC no longer triggers reconnect

Plain English: Sometimes GramJS (the library we use to talk to Telegram) sends back garbled data. This is a transient glitch, not a real problem. The broken code treated every garbled response as a reason to disconnect and restart the full reconnect cycle. Now we just log it and move on.

`noteMalformedRpcResult()` no longer calls `requestReconnect()`. GramJS transient malformed results are silently logged and ignored.

4.3 Fix 4-6: userListener.ts – Reconnect dedup + cycle guard

Plain English: During the storm, multiple errors would fire at once, each telling the listener “reconnect now!” – so it would try to reconnect over itself, making everything worse. Now each reconnect gets a unique ID, and if a reconnect is already in progress, subsequent requests are ignored. If the reconnect fully fails, it waits 60 seconds before trying again.

- **cycleId**: 8-char UUID generated per `requestReconnect()` call, logged in all reconnect messages
- **Cooldown gate**: subsequent reconnect requests within the same cycle are dropped
- **Deferred retry**: `scheduleDeferredRetry()` schedules a fresh reconnect attempt after `authKeyDupDeferredRetryMs()` ms

4.4 Fix 7: `authKeyDuplicatedRecovery.test.ts` – Updated for new defaults

Plain English: The automated tests that check reconnect behaviour were still expecting the old broken values (10 attempts, flat delays). Updated them to match the new safe defaults.

All test expectations updated: `maxAttempts` 10 -> 4, new delay array structure, deferred retry function tested.

4.5 Fix 8: `authService.ts` – Structured auth logging

Plain English: When someone connects their Telegram account, the logs now include a unique ID for that flow, timestamps for each step, and a clear label for what went wrong. This makes it much faster to debug auth issues in the future.

- `logAuthEvent()`: structured log with `correlationId`, timing per step, error categorization
- `authCorrelationId()`: stable ID per auth flow for traceability

4.6 Fix 9-10: `gramjsLogSuppress.ts` (NEW) – Flood-wait noise suppression

Plain English: Telegram rate-limits the listener by saying “slow down, wait X seconds.” GramJS printed this message every single time, flooding our logs with 10,036 identical lines (83% of the entire log) and burying real errors. Now these messages are suppressed and only a single summary line is printed every 60 seconds saying “flood-wait happened 42 times this minute.”

- Monkey-patches `console.log` at import time
- Suppresses lines matching `Sleeping for Xs` on flood wait
- Aggregates counts per 60s window
- Emits one consolidated line: `aggregated_flood_wait count=42 window=60s`
- **Result:** 83% log noise eliminated

4.7 Fix 11: `userListener.ts` – Listener heartbeat

Plain English: Before this fix, there was no way to tell if a listener was alive and working except by watching for errors. Now every listener prints a short health message every 60 seconds saying “I am alive, connected for X seconds, last event happened Y seconds ago.” If the heartbeat stops, we know something is wrong immediately.

- `startHeartbeat()` fires `[telegram-conn] event=listener_healthy` every 60s
- Includes `{uptimeMs, connected, lastEventAgeMs, pendingMessages}` telemetry
- Stopped in `stop()` on shutdown

File Summary

File	Change
<code>worker/src/authKeyDuplicatedRecovery.ts</code>	<code>maxAttempts</code> 10 -> 4, escalating delays, deferred retry
<code>worker/src/authKeyDuplicatedRecovery.test.ts</code>	Tests updated for new defaults
<code>worker/src/userListener.ts</code>	Malformed RPC gated, <code>cycleId</code> + <code>cooldown</code> +deferred retry, heartbeat
<code>worker/src/authService.ts</code>	Structured <code>logAuthEvent()</code> with <code>correlationId</code>
<code>worker/src/gramjsLogSuppress.ts</code> (NEW)	Flood-wait suppression, 60s aggregation
<code>worker/src/index.ts</code>	Imports <code>gramjsLogSuppress</code> first

Verification

- **18/18 tests pass** (`authKeyDuplicatedRecovery` + `gramjsMalformedRpcResultPatch`)
- `npm run build` passes clean (TypeScript compilation)
- **Tests cover:** auth key dup event emission, reconnect delay calculation, max attempt clamping, deferred retry timing, malformed RPC result rejection, patch verification

5. Why the Old Code Did Not Have This Problem

The code at 01a2d913 (PR #43, Jul 23) had:

- **4 reconnect attempts** -> full cycle takes 56s
- **Escalating delays** -> first retry fast (`firstms`), later retries grow (15s, 30s)
- **Deferred retry** -> 60s after exhaustion, re-enters cycle
- **No reconnect on malformed RPC** -> GramJS transient errors don't trigger reconnects

This meant: after a deployment restart, most sessions would reconnect within 30-60s. The 4-attempt cycle completed faster than Telegram's rate-limiter escalated, so flood-waits were short-lived. Sessions that still failed would retry after 60s, by which point the rate-limiter had cooled.

6. Lessons Learned

1. **Reconnect parameters are safety-critical.** Changing `maxAttempts` from 4 to 10 with no escalation multiplied recovery time by 5x without any observable benefit during development.
2. **Never remove deferred retry.** Without it, transient failures become permanent disconnections.
3. **GramJS internal errors should not trigger reconnects.** GramJS is a third-party library with its own transient issues (malformed RPC results, `AUTH_KEY_DUPLICATED` false positives). Reconnect decisions should be based on application-level heuristics.
4. **Flood-wait noise is a liability.** 83% of log lines were harmless rate-limit messages. Suppress them aggressively to keep logs actionable.
5. **Telegram sessions are not truly isolated in a single-worker architecture.** A flood-wait on one session can delay lease renewal for all sessions (single event loop). This is a known constraint, not a bug, but it means reconnect parameters must be conservative.